



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

6 марта 2025 г.

Цель работы

- Определить куб в качестве модели объекта сцены.
- Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта).
- Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без `gluLookAt`). Управление производится интерактивно с помощью клавиатуры и/или мыши.
- Предусмотреть возможность переключения между каркасным и твердотельным отображением модели (`glFrontFace/ glPolygonMode`).

Мой вариант заключался в реализации изометрической проекции

Теория

В ходе выполнения лабораторной работы основное внимание было уделено изучению основных способов реализации проекции. Мой вариант заключался в создании изометрической проекции, которая представляет собой тип параллельной проекции, при которой объект отображается без перспективного искажения, то есть все его стороны выглядят одинаково независимо от удалённости от камеры. Для создания изометрической проекции были использованы последовательные повороты объекта: на 45 градусов вокруг оси Y и на 35.264 градуса вокруг оси X. Эти углы позволяют достичь равномерного отображения сторон куба, что является характерной чертой изометрической проекции. Важным аспектом также является работа с матрицами трансформаций, таких как масштабирование, повороты и сдвиги. Эти преобразования выполняются последовательно, определяя, как будет изменяться положение, масштаб и ориентация объекта в пространстве.

Реализация

Используя ruopengl я описал куб по вершинам, затем создал список определяющий множества вершин задающих грани, а также список с цветами граней. После этого я рисовал статический куб и еще один, который уже откликался на нажатия клавиш, а потом применял к ним изометрическую проекцию.

Код

Листинг 1: Файл main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from math import cos, sin, radians
4
5 # Parameters for rotation, scaling, translation
6 angle_x, angle_y = 0, 0
7 scaleVec = [0.5, 0.5, 0.5]
8 translateVec = [0, 0, 0]
9 translateRatio = 0.1
10 wireframe = False # Mode switch
11
12
13 # Rotation matrix for isometric projection
14 def apply_isometric_projection():
15     glRotatef(35.264, 1, 0, 0) # Rotate 35.264 degrees along X-axis
16     glRotatef(45, 0, 1, 0)    # Rotate 45 degrees along Y-axis
17
18
19 def main():
20     if not glfw.init():
21         return
22     window = glfw.create_window(800, 800, "Isometric Cube", None, None)
23     if not window:
24         glfw.terminate()
25         return
26     glfw.make_context_current(window)
27     glfw.set_key_callback(window, key_callback)
28     glfw.set_scroll_callback(window, scroll_callback)
29
30     glEnable(GL_DEPTH_TEST)
31
32     while not glfw.window_should_close(window):
33         display(window)
```

```

34     glfw.destroy_window(window)
35     glfw.terminate()
36
37 # Define cube using edges and colors for each face
38 def draw_cube():
39     # Colors for each face
40     face_colors = [
41         [1.0, 0.0, 0.0], # Red: back face
42         [0.0, 1.0, 0.0], # Green: front face
43         [0.0, 0.0, 1.0], # Blue: left face
44         [1.0, 1.0, 0.0], # Yellow: right face
45         [1.0, 0.0, 1.0], # Magenta: top face
46         [0.0, 1.0, 1.0], # Cyan: bottom face
47     ]
48
49     # Vertices of the cube (edge list)
50     vertices = [
51         # Back face
52         [-0.5, -0.5, -0.5], [0.5, -0.5, -0.5], [0.5, 0.5, -0.5], [-0.5,
53         0.5, -0.5],
54         # Front face
55         [-0.5, -0.5, 0.5], [0.5, -0.5, 0.5], [0.5, 0.5, 0.5], [-0.5,
56         0.5, 0.5]
57     ]
58
59     # Faces of the cube (each face is a set of 4 vertices, with colors)
60     faces = [
61         [0, 1, 2, 3], # Back face
62         [4, 5, 6, 7], # Front face
63         [0, 4, 7, 3], # Left face
64         [1, 5, 6, 2], # Right face
65         [3, 2, 6, 7], # Top face
66         [0, 1, 5, 4] # Bottom face
67     ]
68
69     if wireframe:
70         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
71     else:
72         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
73
74     # Draw the cube
75     for i, face in enumerate(faces):
76         glBegin(GL_QUADS)
77         glColor3fv(face_colors[i]) # Set color for the face
78         for vertex in face:
79             glVertex3fv(vertices[vertex])

```

```

78         glEnd()
79
80
81 def display(window):
82     global angle_x, angle_y
83     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
84     # glClearColor(1.0, 1.0, 1.0, 1.0)
85
86     # Draw static cube
87     glLoadIdentity()
88     apply_isometric_projection() # Apply isometric projection
89     glTranslatef(1, 0, 0)        # Move static cube to the left
90     glScalef(*scaleVec)
91     draw_cube()
92
93     # Draw rotating cube
94     # Apply transformations
95     glLoadIdentity()
96     glTranslatef(*translateVec)
97     glScalef(*scaleVec)
98     apply_isometric_projection() # Apply isometric projection
99
100    glRotatef(angle_x, 1, 0, 0) # Rotate along X-axis
101    glRotatef(angle_y, 0, 1, 0) # Rotate along Y-axis
102    draw_cube()
103
104    glfw.swap_buffers(window)
105    glfw.poll_events()
106
107
108 def key_callback(window, key, scancode, action, mods):
109     global angle_x, angle_y
110     global translateVec, scaleVec, wireframe
111
112     if action == glfw.PRESS or action == glfw.REPEAT:
113         # Cube rotation
114         if key == glfw.KEY_RIGHT:
115             angle_y -= 3
116         if key == glfw.KEY_LEFT:
117             angle_y += 3
118         if key == glfw.KEY_UP:
119             angle_x -= 3
120         if key == glfw.KEY_DOWN:
121             angle_x += 3
122
123     # Translation (Move by X, Y, Z)

```

```

124         if key == glfw.KEY_W:
125             translateVec[1] += translateRatio
126         if key == glfw.KEY_S:
127             translateVec[1] -= translateRatio
128         if key == glfw.KEY_A:
129             translateVec[0] -= translateRatio
130         if key == glfw.KEY_D:
131             translateVec[0] += translateRatio
132         if key == glfw.KEY_Q:
133             translateVec[2] += translateRatio # Move forward
134         if key == glfw.KEY_E:
135             translateVec[2] -= translateRatio # Move backward
136
137     # Scaling
138     if key == glfw.KEY_Z:
139         scaleVec[0] += 0.1
140         scaleVec[1] += 0.1
141         scaleVec[2] += 0.1
142     if key == glfw.KEY_X:
143         scaleVec[0] -= 0.1
144         scaleVec[1] -= 0.1
145         scaleVec[2] -= 0.1
146
147     # Toggle between wireframe and solid
148     if key == glfw.KEY_M:
149         wireframe = not wireframe
150         if wireframe:
151             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
152         else:
153             glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
154
155     # Close window
156     if key == glfw.KEY_ESCAPE:
157         glfw.set_window_should_close(window, 1)
158
159
160 def scroll_callback(window, xoffset, yoffset):
161     global scaleVec
162     scaleVec[0] += yoffset * 0.1
163     scaleVec[1] += yoffset * 0.1
164     scaleVec[2] += yoffset * 0.1
165
166
167 if __name__ == "__main__":
168     main()

```

Вывод программы

Программа создала окно и отрисовала куба в изометрической проекции. Первый всегда был неподвижен, а второй можно было вращать и переключать режим отображения на каркасный.

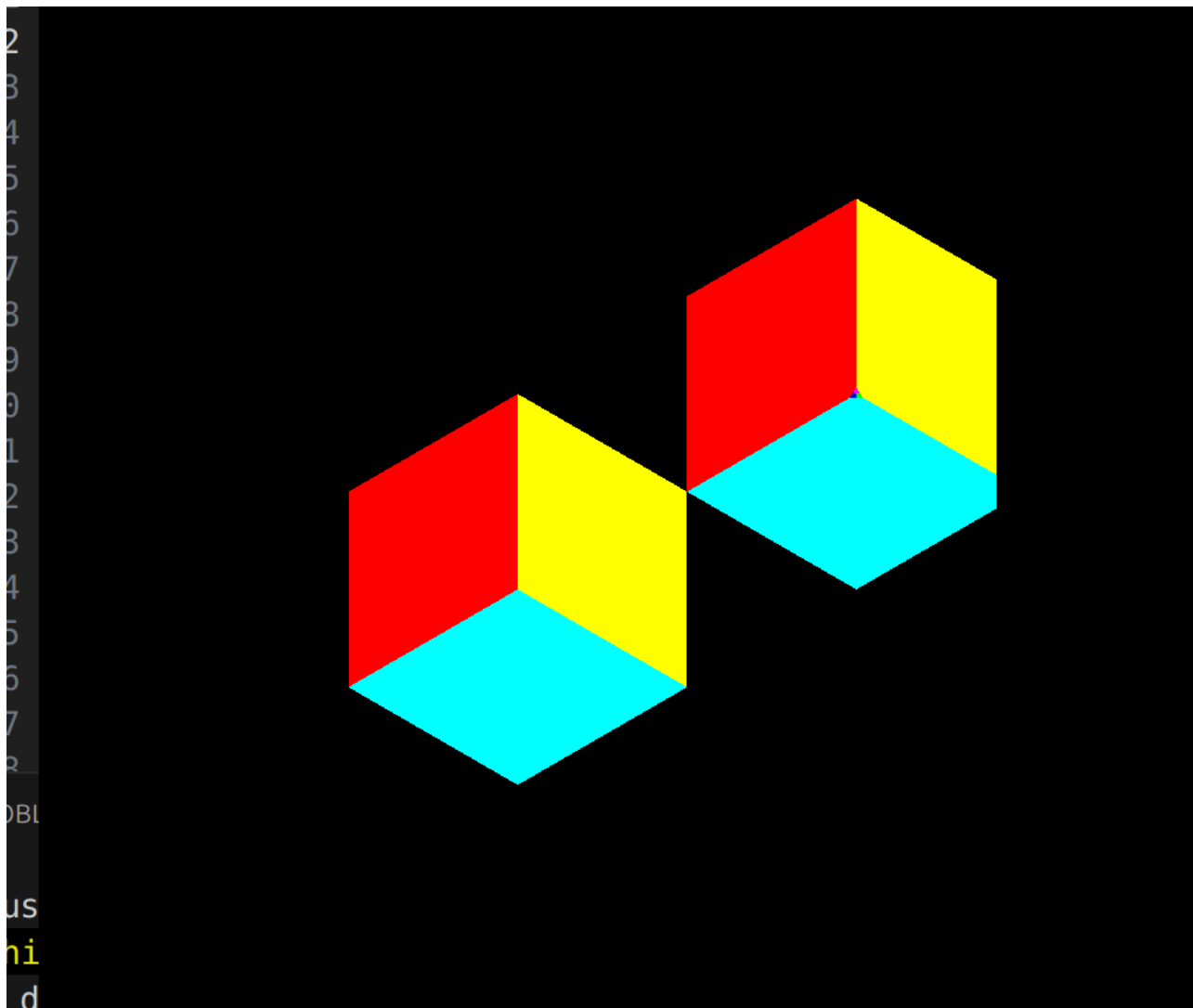


Рис. 1 — Исходная фигура

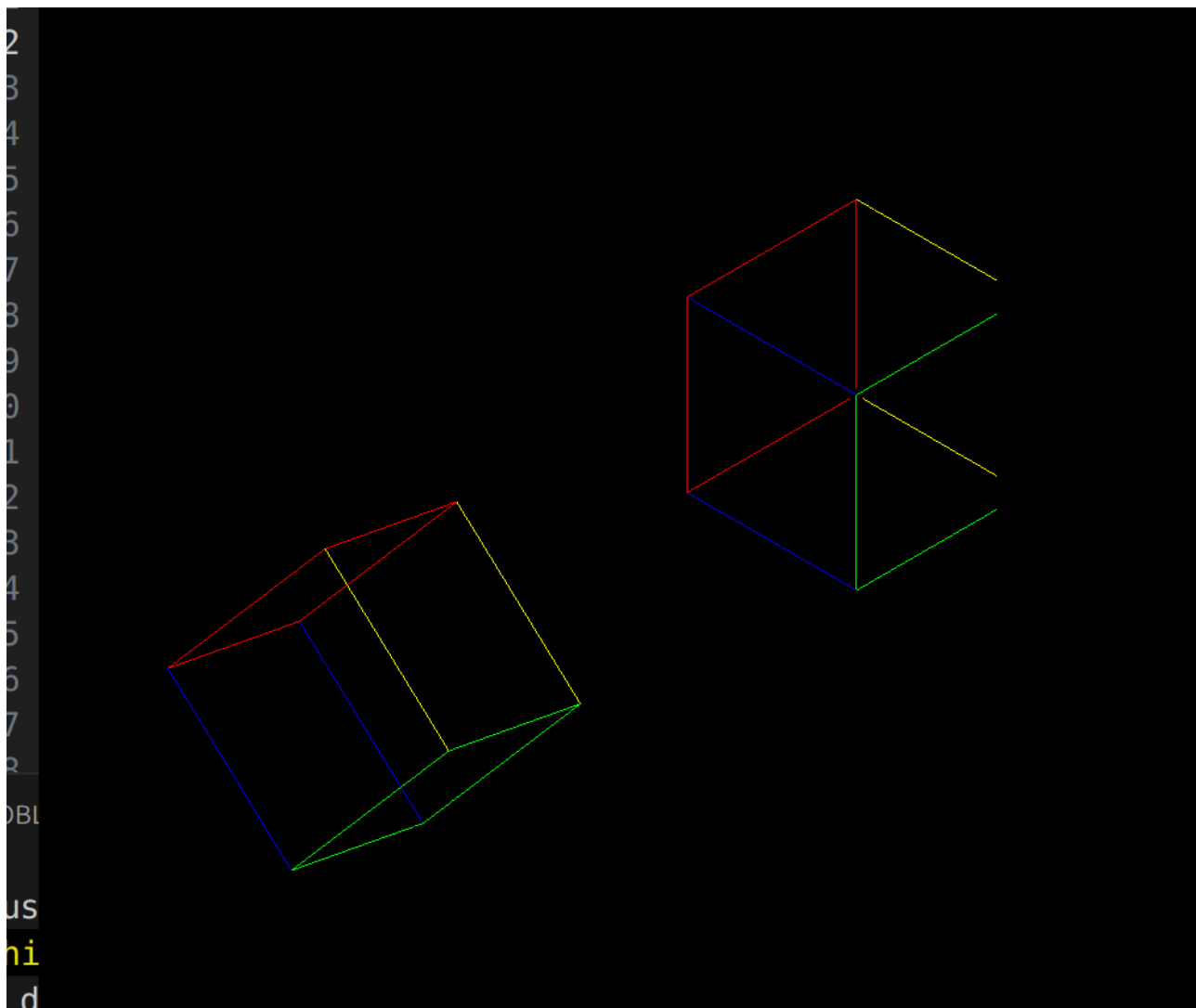


Рис. 2 — Фигура после перемещения

Вывод

В ходе лабораторной работы были изучены основы работы с OpenGL, реализована изометрическая проекция, управление модельно-видовыми преобразованиями, а также переключение между каркасным и твердотельным режимами отображения. Кроме того, я познакомился с принципами матричных проекций и поработал с ними.